

Practical: 2	Implementation and Time analysis of linear and binary search algorithm.
17/08/2026	

Linear Search:-

Aim

To implement the Linear Search algorithm using Python and analyze its time complexity.

Algorithm

1. Start.
2. Read the elements of the array.
3. Read the element to be searched, called the **key**.
4. Start from the first element of the array.
5. Compare the key with the current element.
6. If both are equal, display the position of the element and stop searching.
7. If they are not equal, move to the next element.
8. Continue the process until the key is found or all elements have been checked.
9. If all elements are checked and the key is not found, display "**Element not found.**"
10. Stop.

Program:-

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1  
  
arr = list(map(int, input("Enter elements: ").split()))  
key = int(input("Enter element to search: "))  
  
result = linear_search(arr, key)  
  
if result != -1:  
    print("Element found at index:", result)  
else:  
    print("Element not found")
```

Output:-

```
Enter elements: 10 25 35 45 55 65
```

```
Enter element to search: 45
```

```
Element found at index: 3
```

When element is not present:-

```
Enter elements: 10 25 35 45 55 65
```

```
Enter element to search: 30
```

```
Element not found
```

Analysis:-

Best Case – $O(1)$

The best case occurs when the element being searched is present at the **first position** of the array.

Example:

10 25 35 45 55

If the key is 10, the algorithm compares the key with only the first element and immediately finds it.

- Number of comparisons = **1**
- No further elements need to be checked.
- **Time Complexity = $O(1)$**

Therefore, Linear Search is extremely fast when the required element is at the beginning.

Average Case – $O(n)$

The average case occurs when the element is somewhere in the **middle of the array**, or the position of the element is unknown.

Example:

10 25 35 45 55 65

If the key is 45, the algorithm checks:

10 → 25 → 35 → 45

Several elements have to be compared before finding the required element.

For a randomly positioned element, approximately **$n/2$ comparisons** are required on average.

Therefore:

Time Complexity = $O(n)$

Worst Case – $O(n)$

The worst case occurs in two situations:

1. The element is present at the **last position**.
2. The element is **not present** in the array.

Example:

10 20 30 40 50

If the key is 50, all elements must be checked.

If the key is 60, the algorithm must check every element before concluding that it is not present.

- Maximum comparisons = **n**
- **Time Complexity = $O(n)$**

Space Complexity – $O(1)$

Linear Search does not require any additional data structure or extra array. It only uses a few variables such as the index and search key.

Therefore:

Space Complexity = $O(1)$

Conclusion:-

The Linear Search algorithm was successfully implemented in Python. It searches for an element by checking each element sequentially. It is simple and can be used on both sorted and unsorted data. The algorithm has a best-case time complexity of $O(1)$ and a worst-case time complexity of $O(n)$. Hence, Linear Search is suitable for small or unsorted datasets.

Binary Search

Aim:-

To implement the Linear Search algorithm using Python and analyze its time complexity.

Algorithm:-

1. Start.
2. Read the array elements.
3. Read the element to be searched.
4. Compare the search element with each array element.
5. If a match is found, display its position.
6. If no match is found after checking all elements, display "Element not found".
7. Stop.

Program:-

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1

arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output:-

```
Enter sorted elements: 10 20 30 40 50 60 70
Enter element to search: 50

Element found at index: 4
```

When element is not present

```
Enter sorted elements: 10 20 30 40 50 60 70
```

```
Enter element to search: 35
```

```
Element not found
```

Analysis:-

Best Case – $O(1)$

The best case occurs when the search key is exactly the **middle element** of the array during the first comparison.

Example:

10 20 30 40 50 60 70

Here, 40 is the middle element.

The algorithm calculates the middle position and immediately finds the key.

- Number of comparisons = **1**
- No further searching is required.
- **Time Complexity = $O(1)$**

Average Case – $O(\log n)$

In the average case, the key is found after several divisions of the array.

Binary Search eliminates **half of the remaining elements** after every comparison.

For example:

64 elements

↓

32 elements

↓

16 elements

↓

8 elements

↓

4 elements

↓

2 elements

↓

1 element

The number of steps grows logarithmically with the size of the array.

Therefore:

Time Complexity = $O(\log n)$

This makes Binary Search much faster than Linear Search for large sorted datasets.

Worst Case – $O(\log n)$

The worst case occurs when:

- The element is located at the last possible position after repeated divisions, or
- The element is not present in the array.

Even in this situation, Binary Search keeps dividing the search space into half.

For n elements, the maximum number of divisions is approximately:

$\log_2(n)$

Therefore:

Time Complexity = $O(\log n)$

For example, with 1024 elements, Binary Search needs at most around 10 divisions to reduce the search space to

one element.

Space Complexity – $O(1)$

The given program uses **iterative Binary Search**, so it does not create recursive function calls or additional arrays.

It only uses variables such as:

- low
- high
- mid

Therefore:

Space Complexity = $O(1)$

Conclusion:-

The Binary Search algorithm was successfully implemented in Python. It efficiently searches for an element by repeatedly dividing the sorted array into two halves. It has a best-case time complexity of **$O(1)$** and average and worst-case complexity of **$O(\log n)$** . Therefore, Binary Search is more efficient than Linear Search for large **sorted datasets**.

Practical: 3**Implementation of max-heap sort algorithm****17/08/2026****Max-Heap Sort****Aim**

To implement the Max-Heap Sort algorithm using Python and analyze its time and space complexity.

Algorithm**Max-Heapify**

1. Start.
2. Consider the current element as the largest element.
3. Find the left child using $2 \times i + 1$.
4. Find the right child using $2 \times i + 2$.
5. Compare the left child with the current largest element.
6. Compare the right child with the current largest element.
7. If a child is greater than the current element, swap them.
8. Recursively apply Max-Heapify to the affected subtree.
9. Stop when the Max-Heap property is satisfied.

Heap Sort

1. Start.
2. Read the array elements.
3. Build a Max-Heap from the given array.
4. The largest element will be at the root of the heap.
5. Swap the root element with the last element.
6. Reduce the heap size by one.
7. Apply Max-Heapify to restore the Max-Heap property.
8. Repeat Steps 5–7 until only one element remains.
9. Display the sorted array.
10. Stop.

Program

```
def max_heapify(arr, n, i):  
    largest = i  
    left = 2 * i + 1  
    right = 2 * i + 2  
  
    if left < n and arr[left] > arr[largest]:  
        largest = left  
  
    if right < n and arr[right] > arr[largest]:  
        largest = right  
  
    if largest != i:  
        arr[i], arr[largest] = arr[largest], arr[i]  
        max_heapify(arr, n, largest)
```

```
def heap_sort(arr):
    n = len(arr)

    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        max_heapify(arr, n, i)

    # Extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        max_heapify(arr, i, 0)

arr = list(map(int, input("Enter elements: ").split()))

heap_sort(arr)

print("Sorted Array:", arr)
```

Output

```
Enter elements: 12 11 13 5 6 7

Sorted Array: [5, 6, 7, 11, 12, 13]
```

Analysis

Best Case – $O(n \log n)$

Even when the array is already sorted, Heap Sort still builds a Max-Heap and repeatedly removes the maximum element.

The heap operations require logarithmic time, and the extraction operation is performed approximately n times. Therefore:

Time Complexity = $O(n \log n)$

Average Case – $O(n \log n)$

For randomly arranged elements, Heap Sort first builds the Max-Heap and then repeatedly extracts the largest element.

- Building the heap takes $O(n)$.
- Each deletion from the heap takes $O(\log n)$.
- There are approximately n deletions.

Therefore:

Time Complexity = $O(n \log n)$

Worst Case – $O(n \log n)$

Heap Sort also has $O(n \log n)$ time complexity in the worst case.

Even if the input is arranged in the most unfavorable order, the heap maintains its structure and each extraction requires at most $O(\log n)$ time.

Therefore:

Time Complexity = $O(n \log n)$

Space Complexity – $O(1)$

The given Heap Sort algorithm sorts the elements **in-place**. It does not require an additional array proportional to the input size.

Only a few variables are used during heapification and swapping.

Therefore:

Space Complexity = $O(1)$

Conclusion

The Max-Heap Sort algorithm was successfully implemented in Python. The algorithm uses a Max-Heap to repeatedly select the largest element and place it at its correct position. It has a time complexity of **$O(n \log n)$** in all cases and a space complexity of **$O(1)$** . Therefore, Heap Sort is an efficient and memory-efficient sorting algorithm suitable for large datasets.

Practical: 4	Implementation and Time analysis of factorial program using iterative and recursive method .
17/08/2026	

Factorial Using Iterative and Recursive Methods

Aim

To implement a factorial program using **iterative and recursive methods** in Python and analyze their time and space complexity.

Algorithm

Iterative Method

1. Start.
2. Read a non-negative integer n.
3. Initialize factorial = 1.
4. Repeat from 1 to n.
5. Multiply factorial by the current number.
6. After the loop, display the factorial.
7. Stop.

Recursive Method

1. Start.
2. Read a non-negative integer n.
3. If n is 0 or 1, return 1.
4. Otherwise, calculate:
 $\text{factorial}(n) = n \times \text{factorial}(n - 1)$
5. Continue calling the function until n becomes 1 or 0.
6. Return the calculated factorial.
7. Display the result.
8. Stop.

Program

```
def factorial_iterative(n):
```

```
    fact = 1
```

```
    for i in range(1, n + 1):
```

```
        fact = fact * i
```

```
    return fact
```

```
def factorial_recursive(n):
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    else:
```

```
return n * factorial_recursive(n - 1)
```

```
n = int(input("Enter a non-negative integer: "))
```

```
print("Factorial using Iterative Method:", factorial_iterative(n))
```

```
print("Factorial using Recursive Method:", factorial_recursive(n))
```

Output

```
Enter a non-negative integer: 5
```

```
Factorial using Iterative Method: 120
```

```
Factorial using Recursive Method: 120
```

Analysis

Iterative Method

The iterative method uses a for loop to calculate the factorial.

For example, when $n = 5$:

$$1 \times 2 \times 3 \times 4 \times 5 = 120$$

The loop executes n times.

Time Complexity = $O(n)$

The algorithm uses only a few variables such as fact and i, regardless of the value of n .

Space Complexity = $O(1)$

Therefore, the iterative method is memory efficient.

Recursive Method

The recursive method calculates the factorial by repeatedly calling itself.

For $n = 5$:

```
factorial(5)
```

```
= 5 × factorial(4)
```

```
= 5 × 4 × factorial(3)
```

```
= 5 × 4 × 3 × factorial(2)
```

```
= 5 × 4 × 3 × 2 × factorial(1)
```

```
= 120
```

The function makes approximately n recursive calls.

Time Complexity = $O(n)$

However, every recursive call is stored in the function call stack until the base case is reached.

Therefore:

Space Complexity = $O(n)$

Conclusion

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result and have a time complexity of $O(n)$. The iterative method requires $O(1)$ space, while the recursive method requires $O(n)$ space because of recursive function calls. Therefore, the iterative method is more memory efficient, whereas the recursive method provides a simpler representation of the factorial definition.